

Registro das Funcionalidades

Requisitos funcionais

Id	Descrição	Solicitante	Prioridade
RF01	O sistema deve permitir que o usuário crie um novo projeto de software respondendo a uma série de perguntas simples no terminal, sem precisar configurar nada manualmente.	Product Manager	Alta
RF02	O sistema deve pedir ao usuário que escolha um nome para o projeto antes de começar a criar os arquivos	Product Manager	Alta
RF03	O sistema deve permitir ao usuário escolher a tecnologia principal que será usada no projeto, entre as seguintes opções disponíveis: Node.js + Express, NestJS, Java Spring Boot, Python FastAPI, Next.js e n8n.	Desenvolvedores	Alta
RF04	O sistema deve permitir ao usuário escolher de que forma o projeto será organizado internamente, entre três modelos de estrutura disponíveis: MVC, Clean Architecture ou Hexagonal Architecture	Desenvolvedores	Alta
RF05	O sistema deve oferecer ao usuário a opção de incluir, no projeto gerado, arquivos prontos de configuração para execução em ambiente isolado (Docker), integração contínua (GitHub Actions) e provisionamento de infraestrutura na nuvem (Terraform/AWS).	Gerente do Projeto	Média
RF06	O sistema deve oferecer ao usuário a opção de gerar automaticamente uma documentação explicativa do projeto utilizando inteligência artificial (Google Gemini), dispensando a escrita manual.	Product Manager	Média
RF07	O sistema deve permitir que novas funcionalidades sejam adicionadas a projetos já existentes por meio do comando kybernus add, mesmo que esses projetos não tenham sido criados com o Kybernus	Desenvolvedores	Alta
RF08	O sistema deve permitir a adição de uma página de documentação interativa da API ao projeto via kybernus add swagger, onde os endpoints disponíveis possam ser visualizados e testados diretamente pelo navegador	Desenvolvedores	Média
RF09	O sistema deve permitir a adição de um serviço de cache via kybernus add redis, incluindo a configuração do container Docker correspondente para uso em ambiente de desenvolvimento e produção.	Desenvolvedores	Média

Registro das Funcionalidades

RF10	O sistema deve permitir a adição de suporte a comunicação bidirecional em tempo real via kybernus add websocket, gerando a configuração do servidor e os handlers necessários para o stack selecionado	Desenvolvedores	Baixa
RF11	O sistema deve permitir a adição de Git hooks automatizados via kybernus add husky, incluindo lint-staged e regras de commit linting para garantir padronização do código antes de cada commit.	Desenvolvedores	Baixa
RF12	O sistema deve gerar, via kybernus auth, o boilerplate completo de autenticação JWT para o stack selecionado, contemplando controllers, services, middlewares de proteção de rotas e estrutura de tokens de acesso e refresh.	Patrocinadora	Alta
RF13	O sistema deve permitir que o comando “kybernus deploy” gere os arquivos de configuração de deployment para as plataformas Vercel, Railway, Fly.io e Render, respeitando as convenções de cada provedor conforme o stack do projeto.	Cliente	Média
RF14	O sistema deve permitir que o comando “kybernus doctor” deve inspecionar o ambiente local do usuário, verificando a presença e as versões mínimas exigidas de Node.js, Python, Java e Docker, além de validar a existência das variáveis de ambiente requeridas.	Desenvolvedores	Média
RF15	O comando “init” deve aceitar flags de linha de comando (--name, --stack, --template e --yes) permitindo execução totalmente não-interativa e integração com scripts de automação e pipelines CI/CD.	Cliente	Média
RF16	O sistema deve suportar a flag --template user/repo no comando “init”, permitindo que o usuário utilize um repositório GitHub próprio como template base, substituindo os templates padrão do Kybernus.	Cliente	Baixa
RF17	Ao selecionar a stack n8n, o sistema deve oferecer quatro templates de caso de uso: Default (infraestrutura limpa), AI Assistant (arquitetura webhook + OpenAI), CRM Tracker (integração Google Sheets → HubSpot via schedule) e System Monitor (uptime monitoring com alertas via Discord Webhook).	Cliente	Baixa

Registro das Funcionalidades

RF18	Ao concluir a geração do projeto, o sistema deve exibir um painel de next steps no terminal com os comandos específicos para instalação de dependências (npm install, pip install, mvn install etc.) e inicialização do servidor conforme a stack escolhida.	Cliente	Média
RF19	O sistema deve coletar telemetria anônima de uso para fins de melhoria da ferramenta, sem rastrear dados pessoais identificáveis. O opt-out deve ser possível a qualquer momento via o comando “kybernus analytics --disable”, e o usuário deve ser notificado sobre a coleta na primeira execução.	Desenvolvedores	Alta
RF20	O sistema deve gerar, independentemente da stack, os arquivos de configuração padronizados do projeto: .env com variáveis de ambiente base, configuração do ESLint para análise estática e Prettier para formatação de código.	Cliente	Média
RF21	Para projetos Node.js + Express, o sistema deve gerar configuração com ts-node-dev para hot reloading em desenvolvimento, Prisma ORM como camada de acesso a dados e TypeScript em modo strict.	Cliente	Alta
RF22	Para projetos Python FastAPI, o sistema deve gerar configuração com SQLAlchemy assíncrono como ORM, Alembic para gerenciamento de migrations, Pydantic v2 para validação de schemas e suporte a async/await nativo.	Cliente	Média
RF23	Para projetos Java Spring Boot, o sistema deve gerar configuração com Maven como build tool, Spring Web MVC para exposição de endpoints REST, Spring Data JPA com Hibernate como ORM e compatibilidade com Java 17+.	Cliente	Média
RF24	O sistema deve ser distribuído como pacote npm e executável via comando “npx kybernus@latest” sem instalação global prévia, sempre resolvendo a versão mais recente publicada no registry.	Desenvolvedores	Baixa
RF25	O sistema deve disponibilizar uma landing page pública com a descrição da ferramenta, instruções de instalação e uso, links para a documentação e para o repositório no GitHub.	Marketing	Baixa

Registro das Funcionalidades

Requisitos não-funcionais

Id	Descrição	Solicitante	Prioridade
RNF01	O sistema deve funcionar nos principais sistemas operacionais (Windows, macOS e Linux) sem que o usuário precise fazer configurações específicas para cada um deles.	Patrocinador	Baixa
RNF02	O processo de criação do projeto deve ser guiado por menus visuais no terminal, com uso de ícones e separadores que deixem cada etapa clara, sem que o usuário precise consultar nenhuma documentação para concluir a tarefa.	Desenvolvedores	Alta
RNF03	Todas as funcionalidades disponíveis no sistema devem ser gratuitas, sem cobrança por uso, sem necessidade de criar conta e sem limitações escondidas por plano pago.	Cliente	Alta
RNF04	O sistema deve ser desenvolvido de forma organizada e com testes automatizados, garantindo que erros sejam identificados rapidamente e que qualquer desenvolvedor consiga contribuir com melhorias sem dificuldade.	Desenvolvedores	Média
RNF05	Os dados de uso coletados pelo sistema não devem identificar o usuário de nenhuma forma, e o tratamento dessas informações deve seguir as regras da Lei Geral de Proteção de Dados (LGPD).	Desenvolvedores	Alta
RNF06	Os projetos gerados pelo sistema devem ter uma organização interna clara, com cada parte do código com uma responsabilidade bem definida, facilitando o crescimento e a manutenção do sistema ao longo do tempo.	Cliente	Média